OS - Lab 2

1. Kể tên 1 hệ thống chỉ cho phép một tiến trình thực thi tại một thời điểm thời gian? 1 hệ thống cho phép nhiều tiến trình thực thi đồng thời?

- MS – DOS là hệ điều hành đơn nhiệm, mỗi lần chỉ thực hiện một chương trình tại một điểm thời gian
- Windows, Linux… là hệ điều hành đa nhiệm, có thể thực hiện đồng thời nhiều chương trình.

2. Trong Unix, lời gọi hệ thống nào được sử dụng để tạo tiến trình.

Fork() được sử dụng để tạo tiến trình trong unix.

3. Một tiến trình sẽ bị kết thúc trong các trường hợp nào?

- Tiến trình tự kết thúc: Tiến trình kết thúc khi thực hiện lệnh cuối cùng của nó và yêu cầu hệ điều hành xoá mình đi, bằng cách sử dụng lệnh gọi hệ thống (system call) exit().
- Tiến trình kết thúc do tiến trình khác có quyền kết thúc nó (VD tiến trình cha kết thúc tiến trình con).
  - Gọi system routine abort với tham số là pid của tiến trình cần được kết thúc.
  - Tiến trình cha kết thúc và kéo theo tất cả tiến trình con của nó đều kết thúc.
- Hệ điều hành thu hồi tất cả các tài nguyên của tiến trình kết thúc (vùng nhớ, I/O buffer,...).
- Tắt hệ thống: Nếu hệ thống bị tắt một cách đột ngột, tất cả các tiến trình đang chạy trên hệ thống sẽ bị kết thúc một cách đột ngột.

4. Trạng thái Sẵn sàng của tiến trình là gì?

Ngay sau khi khởi tạo tiến trình, đưa tiến trình vào hệ thống và cấp phát đầy đủ tài nguyên (trừ processor) cho tiến trình, hệ điều hành đưa tiến trình vào trạng thái ready. Hay nói cách khác, trạng thái ready là trạng thái của một tiến trình trong hệ thống đang chờ được cấp processor để bắt đầu thực hiện.

5. Các tiến trình liên lạc với nhau để làm gì?

Các tiến trình cần phải hợp tác và liên lạc với nhau là để chia sẻ thông tin, như dùng chung tập tin, bộ nhớ, hoặc hợp tác hoàn thành công việc.

6. Khi mở một tab mới trên trình duyệt chrome, trình duyệt này có tạo ra tiến trình mới không?

Trình duyệt có tạo ra tiến trình mới.

7. Tiến trình và luồng khác nhau gì?

Tiến trình mang ý nghĩa rộng hơn và lớn hơn so với luồng, luồng là một phần của tiến trình nằm bên trong tiến trình và chịu sự quản lý của tiến trình, một tiến trình có thể có nhiều luồng, cùng hoạt động, chia sẽ code, bộ nhớ,...,của tiến trình, các luồng trong tiến trình có thể hoạt động đồng bộ với nhau, khi thay đổi code của tiền trình hay một luồng thì sự thay đổi đó có tác động lên toàn bộ các luồng khác.

Tiến trình chịu sự quản lý của hệ điều hành với nhiều thông tin phức tạp và chi tiết hơn so với luồng (ví dụ: PCB lưu trạng thái của tiến trình, các hàng đợi), cơ chế điều phối tiến trình của hệ điều hành cũng phức tạp hơn so với việc quản lý luồng của tiến trình, hệ điều hành sử dụng

các hàng đợi(Job queue, Ready quueu, Device queue...) để điều phối tiến trình, trong khi tiến trình sử dụng cơ chế ngủ thông qua các hàm API để thiết lập chế độ hoạt động của luồng (khởi tạo luồng, đặt luồng vào trạng thái ngủ, đánh thức trở lại, kết thúc luồng...). Khi tiến trình kết thúc thì các luồng trong tiến trình cũng kết thúc nhưng tiến trình phải đợi các luồng kết thúc rồi tiến trình mới kết thúc.

8. Tại sao các hệ điều hành hiện đại hỗ trợ môi trường đa nhiệm ?

Các hệ điều hành hiện đại hỗ trợ môi trường đa nhiệm vì: tăng hiệu suất sử dụng CPU và tăng tốc độ xử lý.

9. Sự khác biệt, mối quan hệ giữa tiến trình và tiểu trình ?

Tiến trình là một bộ phận của chương trình đang thực hiện. Tiến trình là đơn vị làm việc cơ bản của hệ thống, trong hệ thống có thể tồn tại nhiều tiến trình cùng hoạt động, trong đó có cả tiến trình của hệ điều hành và tiến trình của chương trình người sử dụng. Các tiến trình này có thể hoạt động đồng thời với nhau.

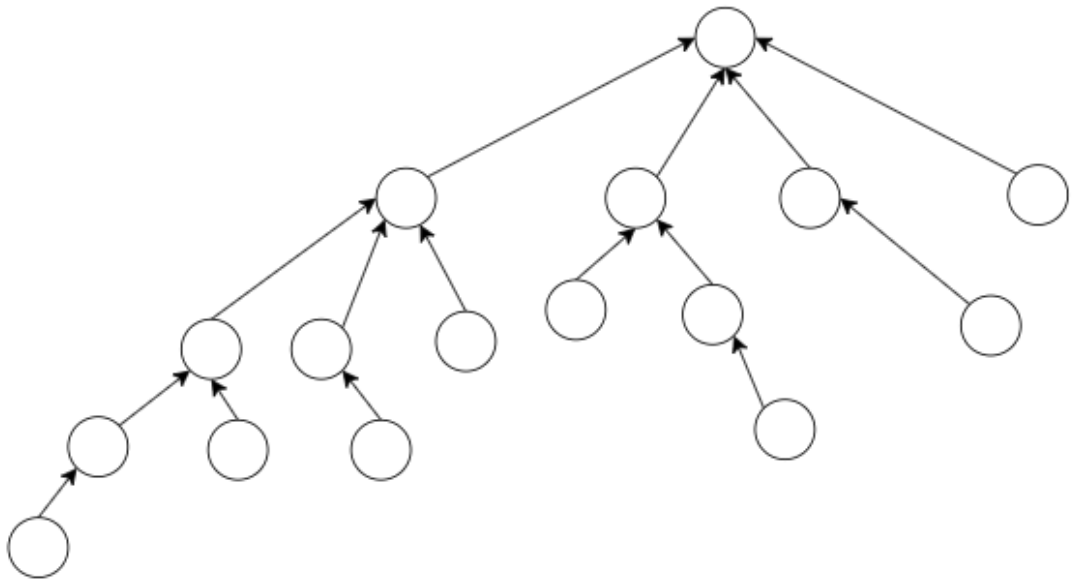
Các hệ điều hành hiện đại sử dụng mô hình đa tiểu trình, trong một tiến trình có thể có nhiều tiểu trình. Tiểu trình cũng là đơn vị xử lý cơ bản trong hệ thống, nó cũng xử lý tuần tự đoạn code của nó, nó cũng sở hữu một con trỏ lệnh, một tập các thanh ghi và một vùng nhớ stack riêng và các tiểu trình cũng chia sẻ thời gian xử lý của processor như các tiến trình. Các tiểu trình trong một tiến trình chia sẻ một không gian địa chỉ chung, điều này có nghĩa các tiểu trình có thể chia sẻ các biến toàn cục của tiến trình, có thể truy xuất đến stack của tiểu trình khác trong cùng tiến trình. Như vậy với mô hình tiểu trình, trong hệ thống có thể tồn tại nhiều dòng xử lý cùng chia sẻ một không gian địa chỉ bộ nhớ, các dòng xử lý này hoạt động song song với nhau.

10. Tìm một số ứng dụng thích hợp với mô hình đa tiến trình; và một số ứng dụng thích hợp với mô hình đa tiểu trình.

- Đa tiến trình: K công nhân để gửi trả ảnh đã chỉnh sửa về cho người dùng.
- Đa tiểu trình: Một server quản lý tập tin thỉnh thoảng phải tự khóa để chờ các thao tác truy xuất đĩa hoàn tất. Nếu server có nhiều dòng xử lý, hệ thống có thể xử lý các yêu cầu mới trong khi một dòng xử lý bị khoá. Như vậy việc thực hiện chương trình sẽ có hiệu quả hơn. Điều này không thể đạt được bằng cách tạo hai tiến trình server riêng biệt vì cầnphải chia sẻ cùng một vùng đệm, do vậy bắt buộc phải chia sẻ không gian địa chỉ.

11. Lệnh sau tạo ra bao nhiêu tiến trình. Hãy vẽ cây tiến trình tạo ra bởi chương trình sau

```
include <stdio.h>
include <unistd.h>
int main()
{
        fork();wait(NULL)
        fork();wait(NULL)
        fork();wait(NULL)
        fork();wait(NULL)
        return 0;
}
```



12. Cho chương trình sau, xác định giá trị của pid tại các dòng A, B, C, và D. Giả sử rằng pid của tiến trình cha la 1500, pid của tiến trình con la 1506

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid, pid1;

//fork a child process
pid = fork();

if (pid< 0) //error occurred
{
        fprintf(stderr, "Fork Failed");
        return 1;
}
else if (pid == 0) //child process
{
        pid1 = getpid();
        printf("child: pid = %d", pid); //line A
        printf("child: pid1 = %d", pid1); //line B
}
else //parent process
{
        pid1 = getpid();
        printf("parent: pid = %d", pid); //line C
        printf("parent: pid1 = %d", pid1); //line D
        wait(NULL);
}
```

```
return 0;
}
```

A: 0
B: 1506
C: 1506
D: 1500

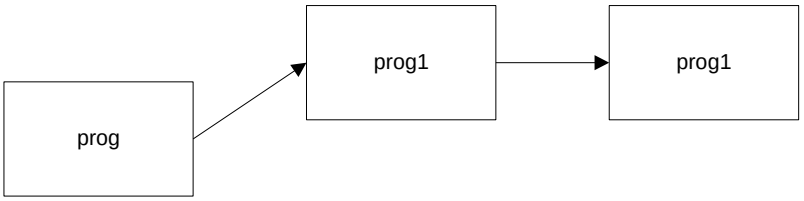
13. Xác định giá trị được in ra tại dòng A

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int value = 5;

int main()
{
        pid_t pid;
        pid = fork();

        if (pid == 0) //child process
        {
                value += 15;
                return 0;
        }
        else (pid > 0) //parent process
        {
                wait(NULL);
                printf("Parent: value = %d", value); //Line A
                return 0;
        }
}
```

=> Giá trị in ra tại dòng A là 20

14. Viết chương trình "prog" tạo ra các cây tiến trình sau

a) Cây tiến trình A




```
#include <stdio.h>

#include <unistd.h>

int main ()

 { fork ();

if (pid ==0) fork () ;

return 0;

 }
```

15. Xét tập các tiến trình sau (với thời gian yêu cầu CPU và độ ưu tiên kèm theo) :

Tiến trình	Thời điểm vào RL	Thời gian CPU	Độ ưu tiên
P_1	0	10	3
P_2	1	1	1
P_3	2	2	3
P_4	3	1	4
P_5	4	5	2

Giả sử các tiến trình cùng được đưa vào hệ thống tại thời điểm 0

a) **Cho biết kết quả điều phối hoạt động của các tiến trình trên theo thuật toán FIFO; SJF; điều phối theo độ ưu tiên độc quyền (độ ưu tiên 1 > 2 > ...); và RR (quantum=2).**
b) **Cho biết thời gian lưu lại trong hệ thống (turnaround time) của từng tiến trình trong từng thuật toán điều phối ở câu a.**
c) **Cho biết thời gian chờ trong hệ thống (waiting time) của từng tiến trình trong từng thuật toán điều phối ở câu a.**
d) **Thuật toán điều phối nào trong các thuật toán ở câu a cho thời gian chờ trung bình là cực tiểu ?**

FIFO

Thời điểm	Tiến trình trong hệ thống	Tiến trình hoàn thành	Tiến trình tiếp theo	WT	TAT
0	P1	-	P1	-	-
10	P2, P3, P4, P5	P1	P2	0	10
11	P3, P4, P5	P2	P3	9	10
13	P4, P5	P3	P4	9	11
14	P5	P4	P5	10	11
19	-	P5	-	10	15

=> WT = 7.6

<u>SJF độc quyền</u>

Thời điểm	Tiến trình trong hệ thống	Tiến trình hoàn thành	Tiến trình tiếp theo	WT	TAT
0	P1	-	P1	-	-
10	P2, P3, P4, P5	P1	P2	0	10
11	P3, P4, P5	P2	P4	9	10
12	P3, P5	P4	P3	8	9
14	P5	P3	P5	10	12
19	-	P5	-	10	15

=> Wt = 7,4

<u>P độc quyền</u>

Thời điểm	Tiến trình trong hệ thống	Tiến trình hoàn thành	Tiến trình tiếp theo	WT	TAT
0	P1	-	P1	-	-
10	P2, P3, P4, P5	P1	P2	0	10
11	P3, P4, P5	P2	P5	9	10
16	P3, P4	P5	P3	7	12
18	P4	P3	P4	14	16
19	-	P4	-	15	16

=> WT = 9

RR (q=2)

Thời điểm	Tiến trình trong hệ thống	Tiến trình hoàn thành	Tiến trình tiếp theo	WT	TAT
0	P1	-	P1	-	-
1	P1, P2	-	P2	-	-
2	P1,P3	P2	P1	0	1
4	P1,P3,P4,P5	-	P5	-	-
6	P1,P3,P4,P5	-	P3	-	-
8	P1, P4, P5	P3	P5	4	6
10	P1,P4,P5	-	P1	-	-
12	P1,P4,P5	-	P5	-	-
13	P1,P4	P5	P1	4	9

15	P1, P4	-	P4	-	-
16	P1	P4	P1	12	13
18	P1	-	P1	-	-
19	-	P1	-	9	19

=> Wt= 5,8

Thời gian chờ của RR là min.

16. Giả sử có các tiến trình sau trong hệ thống :

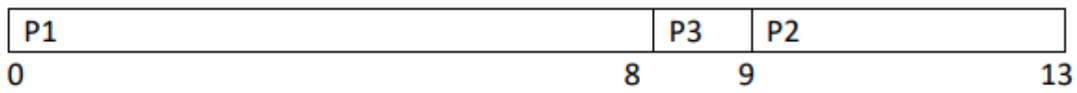
Tiến trình	Thời điểm vào RL	Thời gian CPU
P_1	0.0	8
P_2	0.4	4
P_3	1.0	1

Sử dụng nguyên tắc điều phối độc quyền và các thông tin có được tại thời điểm ra quyết định để trả lời các câu hỏi sau đây:

a. **Cho biết thời gian lưu lại trung bình trong hệ thống (turnaround time) của các tiến trình trong thuật toán điều phối FIFO.**

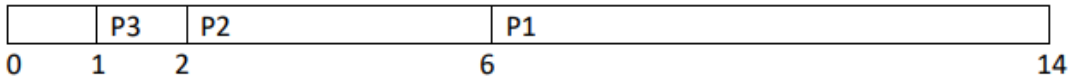
Wt = (8+ 7,6+ 11)/3= 8,87

b. **Cho biết thời gian lưu lại trung bình trong hệ thống (turnaround time) của các tiến trình trong thuật toán điều phối SJF.**



Wt= 9,53

c. **Thuật toán SJF dự định cải tiến sự thực hiện của hệ thống , nhưng lưu ý chúng ta phải chọn điều phối P1 tại thời điểm 0 vì không biết rằng sẽ có hai tiến trình ngắn hơn vào hệ thống sau đó . Thử tính thời gian lưu lại trung bình trong hệ thống nếu để CPU nhàn rỗi trong 1 đơn vị thời gian đầu tiên và sau đó sử dụng SJF để điều phối. Lưu ý P1 và P2 sẽ phải chờ trong suốt thời gian nhàn rỗi này, do vậy thời gian chờ của chúng tăng lên. Thuật toán điều phối này được biết đến như điều phối dựa trên thông tin về tương lai.**



Wt= 6,87

17. Phân biệt sự khác nhau trong cách tiếp cận để ưu tiên cho tiến trình ngắn trong các thuật toán điều phối sau :

a. FIFO.

Độ ưu tiên phục vụ tiến trình căn cứ vào thời điểm hình thành tiến trình. Hàng đợi các tiến trình được tổ chức theo kiểu FIFO. Mọi tiến trình đều được phục vụ theo trình tự xuất hiện cho đến khi kết thúc hoặc bị ngắt. Ưu điểm của thuật toán này là giờ CPU không bị phân phối lại (không bị ngắt) và chi phí thực hiện thấp nhất.

b. RR

Tương tự như định thời FIFO nhưng sự trưng dụng CPU được thêm vào để chuyển CPU giữa các quá trình. Hàng đợi sẵn sàng được xem như một hàng đợi vòng. Bộ định thời CPU di chuyển vòng quanh hàng đợi sẵn sàng, cấp phát CPU tới mỗi quá trình có khoảng thời gian tối đa bằng một định mức thời gian.

c. Điều phối với độ ưu tiên đa cấp

Danh sách sẵn sàng được phân tách thành các danh sách riêng biệt theo cấp độ ưu tiên, mỗi danh sách bao gồm các tiến trình có cùng độ ưu tiên và được áp dụng một giải thuật điều phối thích hợp để điều phối. Ngoài ra, còn có một giải thuật điều phối giữa các nhóm, thường giải thuật này là giải thuật không độc quyền và sử dụng độ ưu tiên cố định

18. Giả sử một hệ điều hành áp dụng giải thuật điều phối multilevel feedback với 5 mức ưu tiên (giảm dần). Thời lượng quantum dành cho hàng đợi cấp 1 là 0,5s. Mỗi hàng đợi cấp thấp hơn sẽ có thời lượng quantum dài gấp đôi hàng đợi ứng với mức ưu tiên cao hơn nó. Một tiến trình khi vào hệ thống sẽ được đưa vào hàng đợi mức cao nhất, và chuyển dần xuống các hàng đợi bên dưới sau mỗi lượt sử dụng CPU. Một tiến trình chỉ có thể bị thu hồi CPU khi đã sử dụng hết thời lượng quantum dành cho nó. Hệ thống có thể thực hiện các tác vụ xử lý theo lô hoặc tương tác, và mỗi tác vụ lại có thể hướng xử lý hay hướng nhập xuất.

a) Giải thích tại sao hệ thống này hoạt động không hiệu quả ?
b) Cần phải thay đổi (tối thiểu) như thế nào để hệ thống điều phối các tác vụ với những bản chất khác biệt như thế tốt hơn ?

Một số vấn đề liên quan đến việc hệ thống hoạt động không hiệu quả như:

Thời gian chờ đợi tăng lên: Do thời lượng quantum tăng theo cấp độ ưu tiên, các tiến trình ở các hàng đợi ưu tiên thấp sẽ có thời gian chờ đợi lâu hơn để được thực thi. Điều này có thể dẫn đến tình trạng chậm trong việc hoàn thành các tiến trình, gây ra sự cản trở trong quá trình xử lý.

Chậm trong việc đáp ứng yêu cầu nhập xuất: Với các tác vụ nhập xuất, việc sử dụng quantum có thể làm gián đoạn quá trình đọc ghi tệp hoặc các hoạt động nhập xuất khác. Điều này có thể gây ra chậm trong việc đáp ứng yêu cầu nhập xuất, ảnh hưởng đến trải nghiệm người dùng và hiệu suất của hệ thống

19. Phân biệt Concurrency (đồng thời) và parallelism (song song)

Concurrency và Parallelism đều giúp tăng hiệu quả xử lý của hệ điều hành, tuy nhiên chúng khác nhau về cách thức và mục đích sử dụng. Concurrency thực hiện đồng thời nhiều tác vụ trên cùng một bộ xử lý, trong khi Parallelism thực hiện nhiều tác vụ đồng thời trên nhiều bộ xử lý khác nhau.

20. Xem xét giải thuật Banker với ma trận yêu cầu cấp phát như sau:

Processes	Allocation A B C	Max A B C	Available A B C
P0	1 1 2	4 3 3	2 1 0
P1	2 1 2	3 2 2	
P2	4 0 1	9 0 2	
P3	0 2 0	7 5 3	
P4	1 1 2	1 1 2	

1. Tính ma trận Need?

2. Kiểm tra xem hệ thống có ở trạng thái an toàn hay không?

3. Xác định tổng của từng loại tài nguyên?

Processes	Allocation			Max (Request)			NEED		
	A	B	C	A	B	C	A	B	C
P0	1	1	2	4	3	3	3	2	1
P1	2	1	2	3	2	2	1	1	0
P2	4	0	1	9	0	2	5	0	1
P3	0	2	0	7	5	3	7	3	3
P4	1	1	2	1	1	2	0	0	0
SUM	8	5	7						
	Available						AVAILABLE		
	A	B	C				A	B	C
	2	1	0				2	1	0
						p4	3	2	2
						p1	5	3	4
						p2	9	3	5
						p0	10	4	7
						p3	10	6	7

Hệ thống ở trạng thái an toàn vì tồn tại thứ tự cấp phát an toàn cho hệ thống là P4, P1, P2, P0, P3.